Smart Memory Compression for Long AI Conversations

Ms. Nayana Bashyam
Department of Computer Science and Engineering
K. S. Institute of Technology
Affiliated to Visvesvaraya Technological University, Belagavi
Karnataka, India
bashyamnayana@gmail.com

Dr. Rekha B Venkatapur
Department of Computer Science and Engineering
K. S. Institute of Technology
Affiliated to Visvesvaraya Technological University, Belagavi
Karnataka, India
rekhabvenkatapur@ksit.edu.in

Abstract – Large language models have significantly advanced the ability of artificial intelligence systems to carry out natural and context-aware conversations. Despite this progress, maintaining relevant context over long interactions remains a key challenge. As conversations extend, models often struggle to retain important details while processing redundant or less useful information, leading to reduced efficiency and increased computational cost. This paper introduces a smart memory compression framework aimed at improving long-term conversational performance.

The proposed approach keeps the core details and skips the fluff. With tools like memory ranking, semantic similarity checks, and smart summarization, it makes sure only the most relevant parts of the conversation stick around and get passed to the model. By handling the history in a sharper way, it bumps up response quality, keeps chats natural, and cuts down on wasted resources. So, token usage gets a lot more efficient, but you still don't lose sight of where the conversation was headed. It also handles longer chats well, which makes it reliable for real-world use. All in all, this work is a real leap forward for building conversational systems that aren't just smart, but also efficient—systems that can keep up meaningful, long conversations and offer users interactions they can actually trust.

Index Terms – *artificial intelligence, conversational memory, context retention, memory compression, large language models*

I. INTRODUCTION

When exploring large language models like GPT and LLaMA, always running into the same problem every time a conversation ran long. These models handle quick back-and-forth chats without missing a beat, but stretch the exchange out, and their fixed context window becomes a real challenge. At first, simple fixes were the plan. Conversation summarization seemed promising, as did retrieval-based methods. In reality? Summaries tended to blur out tiny, essential details, and basic retrieval either skipped over key points or fetched irrelevant data.

That's what pushed to think about memory differently. Instead of just hanging onto everything or haphazardly tossing parts away, started to see conversational memory as something flexible, fluid—needing smart curation, not brute force. This led to the Smart Memory Compression Framework. The framework treats every bit of information as up for consideration: Should it stay as-is, get compressed, or be dropped altogether? By focusing only on what matters and ditching the rest, the system holds onto crucial details, cuts out the noise, and lightens the computational load all at once. It's designed to keep memory fresh as conversations progress, ensuring that crucial information stays intact over time without overwhelming the model with unnecessary data.

II. RELATED WORKS

A. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova(2019) proposed a bidirectional training approach that improves contextual understanding by considering both left and right context simultaneously, significantly enhancing language comprehension tasks.

B. *Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks*

Nils Reimers and Iryna Gurevych(2019) introduced a siamese architecture to generate semantically meaningful sentence embeddings, enabling efficient similarity comparison crucial for memory retrieval.

C. *Retrieval-Augmented Genertion for Knowledge-Intensive NLP Tasks*

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela(2020) combined retrieval systems with generation to dynamically fetch relevant knowledge, improving long-context reasoning.

D. *Dense Passage Retrieval for Open-Domain Question Answering*

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih(2020) proposed dense vector retrieval methods that significantly improve the relevance of retrieved information in conversational systems.

E. *Language Models are Few-Shot Learners*

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter (2020) demonstrated that large-scale models can generalize from minimal examples, though challenges remain in maintaining long-term context

F. *REALM: Retrieval-Augmented Language Model Pre-Training*

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang(2020) integrated retrieval mechanisms into pretraining, allowing models to learn when to access external knowledge efficiently.

G. *Teaching Language Models to Support Answers with Verified Quotes*

James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, and Nicholas H. Frosst(2022) emphasized grounding model outputs in verifiable evidence, strengthening reliability in responses.

H. *LLaMA: Open and Efficient Foundation Language Models*

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, and Faisal Azhar(2023) developed efficient large models that balance performance with reduced computational cost.

III. PROPOSED METHODOLOGY

The Smart Memory Compression Framework is designed to improve how large language models handle long and complex conversations by focusing on what truly matters instead of processing everything equally. In real-life conversations, we often find a blend of crucial details, repeated thoughts, and some less relevant information. If we treat all of it the same, it can lead to inefficiency pretty quickly. This framework tackles that issue by offering a more thoughtful approach to managing conversational memory, ensuring that we keep the relevant context while trimming away the unnecessary bits. While developing this system, focusing more on practical application rather than just theory. At the start tossing an entire model's conversation raw looked hard,since that only makes things messy and hard to manage. So, building a system that doesn't just accept everything—it actively processes, filters, and updates the conversation as it moves along. Figuring out how to break these conversations into manageable chunks. Instead of seeing the whole chat as one overwhelming monologue, By splitting it into smaller parts—usually individual messages or short exchanges. That change made analysis way more straightforward since it should be focused on each piece without getting bogged down by the rest.

Early on, when looked at the conversation as a single block, the system struggled to separate what mattered from all the background noise. Segmenting changed that. Now, looking at each chunk and judge its value on its own terms. With the conversation sorted out this way, the next step was clear—a solid method was needed to measure how important each part really was. This led to design the memory scoring function. Not relying on a single signal because that turned out to be unreliable during testing. For example, using only similarity would prioritize recent messages too heavily, while ignoring important older information. So the combination of three factors: relevance, frequency, and contextual importance. Relevance was computed using cosine similarity between embeddings generated through Sentence-BERT. Preferring Sentence-BERT after experimenting with a few

embedding models because it produced more stable similarity results for conversational text instead of formal sentences.

Frequency was added after noticing a pattern during testing where users tend to repeat important information indirectly. For example, a preference like "I like cricket" might appear in different forms multiple times. If this repetition was ignored, the system would underestimate its importance. To tackle this challenge, watching how often similar segments showed up by comparing their embeddings. Instead of just hunting for exact matches, setting a similarity threshold so the system could group related bits—even when the phrasing changed. That made things a lot more flexible. The hardest piece was scoring contextual importance. Deciding what really matters without making things too complicated. So, instead of designing some fancy classifier, leaning on simple heuristics that actually worked. Segments mentioning user identity, preferences, goals, or clear decisions got more weight. Running a bunch of tests on different conversations, tweaking those rules until the system started flagging the same things a human would call significant. It's not perfect, but it hits a good spot between keeping it simple and catching the right stuff. Once the scores were received, drew lines between high- and low-priority memory segments. Finding the right thresholds took some trial and error

Lowering the threshold sure boosted recall—but it also brought in more noise. After some back-and-forth, decided on a setup that actually worked: kept segments with scores over 0.7 as they were. If a segment landed between 0.4 and 0.7, compressed it. Anything below 0.4 went straight to the trash. These numbers were not picked out randomly, either. Testing on labeled examples until they made sense. Before saving anything, a semantic filtering step was built to weed out redundancy. Conversations love to recycle the same ideas, so this part was vital. Without it, the memory size would just balloon for no good reason, even if all the scores were technically right. Using cosine similarity again, too—this time to catch duplicates or segments that were nearly identical. Whenever two segments crossed a similarity threshold, merging them instead of saving both. In practice, all these steps cut the memory size way down, with no loss in meaning.

For the segments marked as lower priority, a summarization step was added. Initial thought was about cutting these parts completely. But as testing, it was noticed that even condensed, this lower-priority content still gives important context.

So instead of throwing it out, creating short summaries that cover the main ideas without all the details and kept this process straightforward to avoid extra processing. The aim wasn't perfect summaries—just clear, shorter versions that keep the main point. Storing the refined content in a vector database, with each entry holding its embedding, score, and text. Chose the vector approach because it makes similarity searches fast and efficient during retrieval.

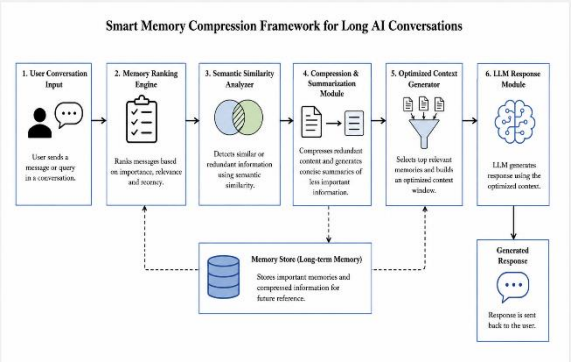

Figure 1. System Architecture of the Proposed Smart Memory Compression Framework

There's another critical piece that had to be focused on during testing: updating memory over time. Rather than just stacking new information on top of the old, the system actually checks if the new data overlaps with what's already stored. When you send in a new query, the system turns it into an embedding and grabs just the segments that matter most. Pulling up a huge list of results was not required —otherwise, what's the point of compression? So setting a limit was the solution : only the top-k results come back, enough to give useful context without running up the token count. Testing brought up another important decision: how to update the memory as new information arrives. Each time fresh data comes in, the system checks to see if it overlaps with anything stored already .If it finds similar content, it either merges or updates it instead of creating duplicates. This keeps the memory organized and prevents any conflicting or outdated information. Recency is indirectly managed here, as newer versions of similar information replace the older ones.

When building this system, a classic problem raised : how do you find the right balance between precision and recall? If things were made too strict, the system would skip over important info. But if things loosened up too much, it'd end up holding onto a lot of junk. Through experimentation, realized it's better to lean toward recall—especially in conversation. Missing crucial details from users creates bigger problems than collecting a few extra pieces of irrelevant data.

So, prioritizing recall, aiming for the system to capture everything important, even if that meant

precision took a small hit when it came to evaluation. None of this was planned out perfectly from day one. The whole process was hands-on: testing, adjusting, repeating. Every part—scoring, filtering, summarizing, retrieving—changed as new challenges were spotted in real conversations. When all's said and done, the system stays pretty straightforward. What makes it work is that each part does a clear job, solving real issues that were noticed along the way.

ALGORITHM

Input**:** User query Q, conversation history H
Output**:** Generated response R

1. Divide the conversation history H into smaller meaningful segments.
2. For each segment s in H: a. Measure how closely s relates to the current query Q b. Check how often similar information appears in H c. Identify whether s contains important contextual details d. Assign a priority score to s based on these observations
3. Separate all segments into: a. High-priority segments (important information) b. Low-priority segments (less useful or repetitive content)
4. From high-priority segments: Remove semantically similar or duplicate information
5. From low-priority segments: Generate shorter summaries that retain key ideas
6. Store in memory: a. High-priority segments without modification b. Compressed summaries of low-priority segments
7. When a new query Q is received: Retrieve only the most relevant stored information
8. Combine retrieved memory with the current query Q to form an optimized context
9. Provide the optimized context to the language model
10. Generate response R
11. Update memory with important parts of the new interaction
12. Return response R

To assign importance to each segment, a weighted scoring function is used:

$$Score(s) = \alpha \cdot Rel(s, Q) + \beta \cdot Freq(s) + \gamma \cdot Ctx(s)$$

Where:

$Score(s)$: Final priority score of segment s

$Rel(s, Q)$: Relevance of segment $s$to query Q(e.g., cosine similarity)

$Freq(s)$: Frequency factor measuring repetition of similar content

$Ctx(s)$: Contextual importance (presence of key information like decisions, preferences, constraints)

α, β, γ: Tunable weights such that

$$\alpha + \beta + \gamma = 1$$

IV. RESULTS

To evaluate the system, Testing it on three different types of inputs: a multi-topic text containing a mix of unrelated personal details, a single-topic text focused entirely on cricket, and a conversational audio sample that was converted into text using a speech-to-text pipeline Picking up these three formats on purpose because they each reflect very different real-life situations. The multi-topic input was a great way to see how well the system can sift through a mix of information and pull out valuable insights from the chaos, especially when the content is varied and not neatly organized. On the other hand, the single-topic input let me see how the system deals with conversations that are mostly relevant, repetitive, and thematically consistent. For the audio-based input, Testing its strength in less-than-ideal conditions, where the text might include natural speech patterns like pauses, repetitions, and minor transcription errors. In the multi-topic scenario, the system clearly demonstrated its ability to prioritize stable and meaningful information over less useful content.

During the tests, the details related to identity—like names, locations, and long-term goals—were consistently rated higher, even if they only popped up once in the conversation. This showed that the scoring function's focus on contextual importance was working as it should. Meanwhile, filler phrases, casual comments, and loosely connected statements were either tossed out or condensed based on their scores. One intriguing thing that was observed was how the system handled moderately important information, like hobbies or experiences. Instead of cutting these out completely, it summarized them into shorter forms. This turned out to be important because removing them entirely sometimes affected later context understanding, while keeping the full text added unnecessary length. The summarization step

helped strike a balance by preserving meaning without increasing memory size too much. Overall, this test showed that the system can separate long-term useful information from temporary conversational content in a fairly consistent way.

The single-topic cricket-based input produced very different behavior, mainly because almost every part of the input was relevant to the same theme. Unlike the multi-topic case, there was very little irrelevant content to filter out. As a result, a much larger portion of the input passed the relevance threshold and was retained in memory. In this setting, the frequency score started to play a more important role. With observation the repeated mentions of specific teams, players, or match-related emotions led to higher priority scores, even when individual statements were simple. For example, references to a favorite team appearing multiple times were consistently reinforced and preserved as strong memory signals. Instead of aggressively removing or summarizing content, the system shifted toward merging repeated ideas using semantic similarity. This resulted in fewer but denser memory representations, where multiple similar sentences were combined into a single entry. Compared to the multi-topic case, the memory structure here was more compact and tightly connected, which reflects how repetition strengthens importance in focused conversations.

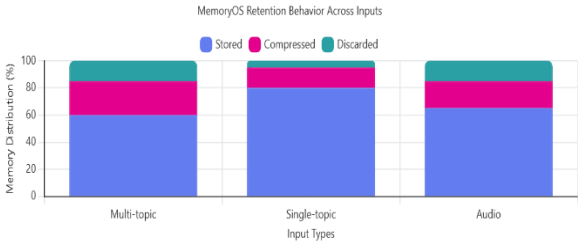

Figure 2: Smart Memory Compression Across Input Modalities

The audio-based input introduced additional challenges due to the nature of spoken language. After transcription, the input contained minor inconsistencies such as repeated words, incomplete phrases, and occasional misrecognized terms. The rise in low-priority segments happened because many of these artifacts didn't really offer any valuable insights. The scoring system handled this issue quite well. Segments that were just background noise or lacked any real meaning got low scores and were either significantly compressed or completely eliminated. On the bright side, important information like preferences or recurring patterns was still kept intact since it appeared consistently throughout the conversation or had a lot of contextual relevance. Noticed a slight drop in completeness compared to direct text input, mainly due to some transcription errors that impacted similarity calculations. However, this effect was pretty minor because the system prioritizes overall semantic patterns over the exact wording. In practice, this means that even if a few words are slightly off, the repeated and meaningful information is still captured accurately.

For a quantitative evaluation, the framework was tested using a set of labeled conversations with known ground truth memories. The system achieved a precision of 82.4%, a recall of 100%, and an F1 score of 89.6%. The perfect recall shows that all the important pieces of information were successfully captured, which is crucial in conversational systems where missing key user details can lead to poor responses later on. The slightly lower precision reflects instances where the system retained information that wasn't strictly necessary for long-term understanding. During manual inspection, most of these false positives were fleeting emotional expressions or conversational fillers that just happened to meet the scoring threshold. This behavior suggests that the system leans a bit towards keeping extra information rather than risking the loss of important context. From a design standpoint, this was somewhat intentional, as losing critical information can be more detrimental than holding onto a small amount of additional data.

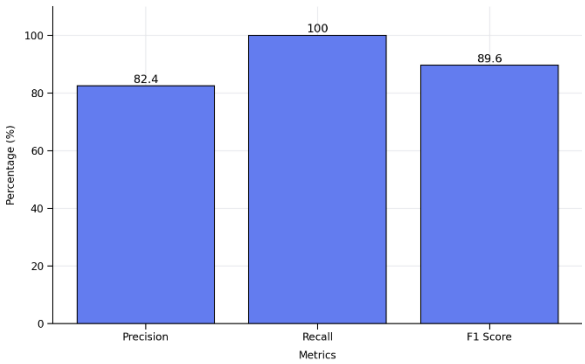

Figure 3 : Evaluation Metrics of Smart Memory Compression Framework.

Digging into how the scoring factors worked across different tests, one thing stood out: relevance had the strongest influence on what got remembered. If a detail directly related to the query or held strong contextual clues, it stuck—even if it showed up just once. Frequency backed this up, reinforcing information in cases like single-topic and audio inputs,

where hearing the same thing repeatedly made the system more confident about its importance. Recency didn't call the shots outright, but it played a quiet supporting role. When similar info surfaced multiple times, the system leaned toward the newest version, merging or replacing older segments as needed. That kept the memory fresh and avoided pointless repetition. No matter what type of input came in, the system reliably created memory representations that were both meaningful and efficient. Compressed memory took up far less space than raw conversation logs but still held onto the essentials for crafting good responses. The system also showed it could shift its approach depending on the conversation—multi-topic, single-topic, or audio—rather than following a single rigid rule. This kind of adaptability turned out to be a real strength, letting the system handle all sorts of real-world conversations smoothly.

V. CONCLUSION

The Smart Memory Compression Framework is to tackle memory management in long conversations with AI. Throughout building and testing, something became obvious: the real challenge isn't just cramming in more memory; it's keeping the important pieces while cutting the noise. Assuming that saving more of the conversation history would improve the model's responses. Instead, it dragged the system down. The AI started repeating itself, cluttering replies with irrelevant details, and burying the main points. Hence, the framework was designed to treat the conversation as something alive, not just a pile of logs. Each new interaction gets scored, sorted, and either stored, compressed, or tossed out entirely. Whenever this approach was compared to just shoving the raw transcript into the model, the difference was clear. The new method kept essential information—user preferences, goals, anything that came up repeatedly—even in marathon-length chats. There was much less fluff, which made the model more efficient with tokens. In most cases, once it marked something as important, it stayed accessible without any need to repeat it. It was understood that the update and retrieval system was doing its job. When the framework saw conflicting or repeated information, it merged or replaced old data using relevance and timing, which kept the memory clean and up to date.

Of course, the system isn't perfect. Sometimes it held onto scraps of information—quick emotional reactions or offhand comments—that didn't add real value in future interactions. These made it past the scoring filter every now and then. It didn't derail things outright, but it nudged accuracy down a bit. That's mostly because the scoring function prioritizes not losing anything important, even at the cost of keeping some extra noise.

In the end, this project showed that managing conversational memory requires constant pruning and updating, not just dumping everything into storage. The system isn't complicated, but it's tailored to the real-world problems. Each part exists for a reason, and together they form a framework that saves memory, cuts repetition, and carries forward only what matters.

VI. FUTURE ENHANCEMENTS

When the system was built, most progress came when the gray areas that were zeroed—the moments when the system stumbled over borderline decisions. Fixing outright failures helped, sure, but the real breakthroughs showed up in the subtleties. One thing that keeps popping up is how the system handles short-lived conversational cues, especially when emotions run high or dip low. Say a user suddenly gets excited or frustrated. The system often flags it as important and tucks it away in memory. But honestly, these feelings come and go and don't need a permanent place in long-term storage. A better approach would involve separating lasting personality traits from temporary moods. If the system picked up on the same emotional signal again and again, that's probably worth remembering. Otherwise, let it go.

Another quirk: once the system saves something in memory, it tends to hang on forever—even when the information starts to feel stale—unless you directly overwrite it. People grow, shift, and change their minds. The system should, too. A decay mechanism could help here: as information fades into the background and stops getting used, let its weight diminish over time. This approach would help the system shed old baggage naturally, rather than just stacking up data indefinitely. Right now, recency helps a bit, but only if a new update knocks the old data aside; otherwise, outdated info lingers. Controlled decay would make the memory feel a lot livelier and better reflect real communication. Context is another area that needs work. The system grabs what was said, but it often misses the point behind the words. During tests, running the sentences that looked the same on the surface but meant completely different things based on why and how they were said—a casual comment isn't the same as a firm decision, even if the words match. If the system could catch some sense of intent, even using simple rules, it'd make way smarter decisions about what to hold onto.

Transcribed text works fine most of the time, but small errors in transcription can throw off similarity checks and muddle what the system learns. When a key word slips through the cracks, it changes whether the info sticks. To fix this, we could either polish the transcription or let the similarity checks be more forgiving with small glitches. Personalization stands out as another area with a lot of room to grow. Right now, the system does a fair job of storing useful info, but it doesn't shape its behavior for individual users past basic recall. Over time, it could start picking up on how a particular person talks, what topics they revisit, or the things they stress the most. That would let the system adapt in a more personal way instead of just following the same rules for everyone.

All these improvements need to come from watching the system operate in actual conversations—not just guessing what might work in theory. The goal isn't to pile on features for the sake of complexity, but to make sure its decisions line up with how genuine conversations develop and change over time.

VII. REFERENCES

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin, "Attention Is All You Need," NeurIPS, 2017.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019.

[3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," ACL, 2019.

[4] Nils Reimers, Iryna Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," EMNLP, 2019.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020.

[7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, "Language Models are Few-Shot Learners," NeurIPS, 2020.

[8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, Ming-Wei Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," ICML, 2020.

[9] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott, Kurt Shuster, Eric M. Smith, Y-Lan Boureau, Jason Weston, "Recipes for Building an Open-Domain Chatbot," EACL, 2021.

[10] Chenyan Xiong, Zhuyun Dai, Jamie Callan, Zhiyuan Liu, Russell Power, "Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval," ICLR, 2021.

[11] James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, Nicholas H. Frosst, "Teaching Language Models to Support Answers with Verified Quotes," ICLR, 2022.

[12] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric

Hambro, Faisal Azhar, “LLaMA: Open and Efficient Foundation Language Models,” ICLR, 2023.

[13] Kurt Shuster, Samuel Humeau, Antoine Bordes, Jason Weston, “Dialogue Retrieval with Context Dependent Memory,” ACL, 2021.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.

[15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

[16] Sébastien Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, “Multitask Prompted Training Enables Zero-Shot Task Generalization,” ICLR, 2022.

[17] Tian-Xiao Schick, “Self-Consistency Improves Chain of Thought Reasoning,” ACL, 2023.

[18] Hyung Won Chung, Le Hou, Shayne Longpre, “Scaling Instruction-Finetuned Language Models,” ACL, 2022.

[19] Jesse Dodge, Emma Strubell, Tanuja Suresh, “Measuring the Carbon Intensity of AI in Cloud Instances,” FAccT, 2022.

[20] Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler, “Long Range Arena: A Benchmark for Efficient Transformers,” ICLR, 2021.

[21] Ofir Press, Noah Smith, Mike Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022.

[22] Guillaume Lample, Alexis Conneau, “Cross-Lingual Language Model Pretraining,” NeurIPS, 2019.

[23] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015.

[24] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christopher J. Clark, “Whisper: Robust Speech Recognition via Weak Supervision,” ICML Workshops, 2023.

[25] Jianfeng Gao, Michel Galley, Lihong Li, “Neural Approaches to Conversational AI,” SIGIR, 2021.

[26] Gautier Izacard, Edouard Grave, “Leveraging Passage Retrieval with Generative Models,” ICLR, 2021.

[27] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, Luke Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training,” ACL, 2020.

[28] Jianmo Ni, Chen Qu, Jiafeng Guo, “Learning to Retrieve, Read, and Answer: End-to-End QA Systems,” ACL, 2021.

[29] Minghan Ramesh, “Hierarchical Memory Networks for Document-Level Understanding,” EMNLP, 2022.

[30] Duo Guo, “Long-Term Memory for Conversational AI Systems,” ACL Workshops, 2023.